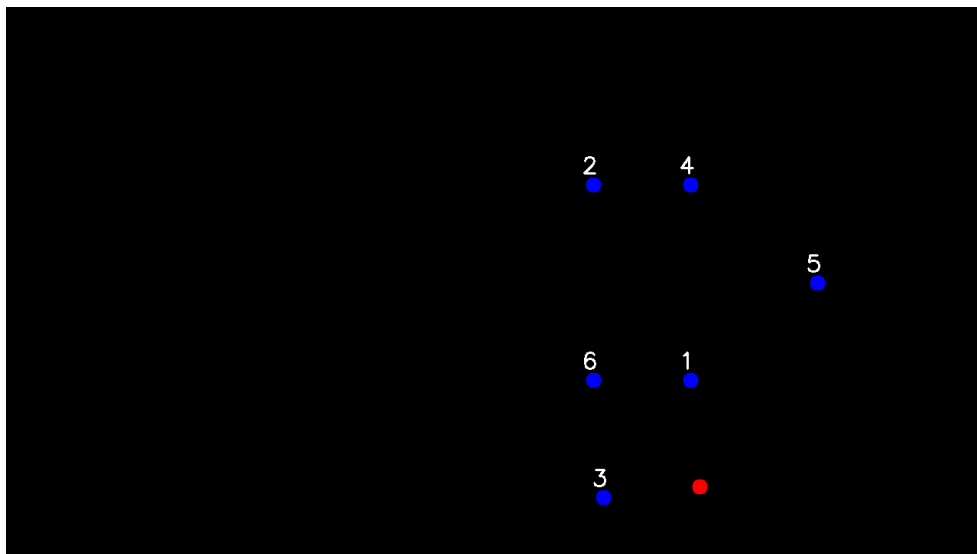


Task 3: Fundamentals of 3D Computer Vision

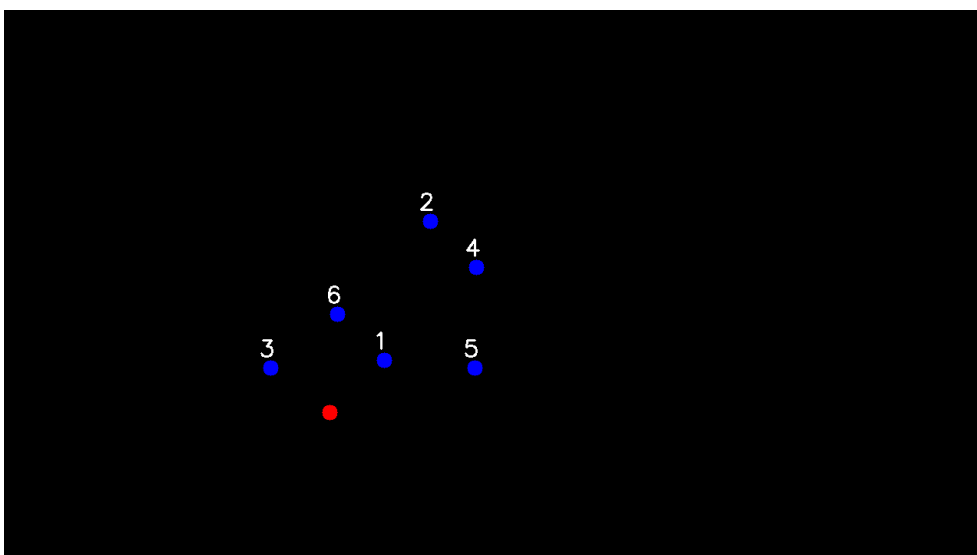
Two different cameras, each with an unknown camera intrinsics and extrinsics, are observing a static structure with 7 unique feature points. Assume that the 2d-to-2d corresponding points between both camera images have been identified and that the 3D position of 6 (blue) feature points are known. The 3D position of the single feature point marked in red is unknown.

For this task, assume that all 3D positions are with respect to an arbitrary world frame in ENU coordinates. (0,0,0) would represent the origin in this world frame. (1,0,0) would represent 1 meter to the East of the world frame origin.

The following is a visualisation of the 7 feature points observed by both cameras.



Feature points detected from camera #1



Feature points detected from camera #2

Both cameras produce a 1280x720px image but may have different intrinsic matrices. We follow openCV's convention for pixel position, where [0,0] is top-left of the image.

Given the world coordinates and pixel coordinates of the feature points are as follows:

Name of Feature Pt	3D World Coordinate (X,Y,Z)	Pixel Position (X,Y) in Image from Camera #1	Pixel Position (X,Y) in Image from Camera #2
Blue #1	[1.0, 2.0, 1.0]	[894, 487]	[494, 455]
Blue #2	[0.0, 0.0, 1.0]	[767, 232]	[554, 274]
Blue #3	[0.0, 3.0, 0.0]	[780, 640]	[346, 465]
Blue #4	[1.0, 0.0, 1.0]	[894, 232]	[614, 334]
Blue #5	[2.0, 1.0, 0.0]	[1060, 360]	[612, 465]
Blue #6	[0.0, 2.0, 1.0]	[767, 487]	[433, 395]
RED	???	[906, 626]	[423, 523]

Your code should find (1) the XYZ position of the last (Red) feature point, (2) the intrinsic matrices, (3) rotation matrix and (4) XYZ position of both cameras. Your answer for (1) and (4) should be in metres with respect to the world frame in ENU coordinates. In your submission for this task, please include your code and the answer to the above in a txt file. Please also include instructions on how to compile (for C++17) and run your code.

Your solution is expected to be within a L2 Norm tolerance of 15cm distance and tolerance of 20 pixel units for focal length is acceptable.

If your solution is written in python3, you are only allowed to use numpy packages. If your solution is written in C++17, you are only allowed to use Eigen and OpenCV for vector/matrix math operations only. Additionally, you are **NOT allowed to use any functions listed** in the Camera Calibration and 3D Reconstruction module of OpenCV. (See https://docs.opencv.org/4.x/d9/d0c/group_calib3d.html)